



NGO OS

A Composable Operating System for Non-Governmental Organizations

"Opinionated about primitives. Unopinionated about workflows."

Concept Document • March 2026

Contents

1. The Problem
2. Core Philosophy
3. The Primitive Layer
4. Groups: Flexible Organizational Structure
5. Tags: Cross-Cutting Descriptors
6. The Composition Layer
7. The Form Builder
8. Platform Services
9. Competitive Landscape
10. Business Model & What's Next

1. The Problem

NGOs are wildly diverse. A rural health clinic in Bihar, a global climate advocacy group, and a local after-school program all have fundamentally different needs. Yet they are forced to choose between tools that are either too rigid (vertical SaaS that locks them into one domain), too complex (enterprise CRMs repurposed for nonprofits), or too fragile (spreadsheets and manual processes that break at scale).

The result: massive duplication of effort, poor data quality, inability to report impact credibly, and an enormous gap between what technology could enable and what NGOs actually use.

2. Core Philosophy: Primitives, Not Prescriptions

Instead of building pre-defined modules like “donor management” or “program tracker,” NGO OS provides a small set of **composable building blocks** that organizations wire together to match how *they* actually work. Think Notion meets Salesforce meets Airtable — but purpose-built for the social sector.

The system is opinionated about the **primitives** (what the building blocks are) but completely unopinionated about **workflows** (how any particular NGO arranges them). This is the foundational design principle.

3. The Primitive Layer

Six core objects form the foundation of NGO OS. Everything the system does is built on combinations of these primitives.

1. Entities — The Universal "Thing" Object

A beneficiary, a donor, a volunteer, a partner org, a school, a village. Rather than separate databases for each, you have one flexible entity system with custom types and fields. An NGO defines its own entity types (“Farmer,” “Self-Help Group,” “Watershed”) and the relationships between them. This is the single most important design decision — it means you are not forcing an education NGO to use a model designed for a health NGO.

2. Activities — Any Unit of Work

A training session, a site visit, a distribution event, a counseling session, a survey. Activities connect entities to each other in time and space. They capture *what happened, with whom, where, and when*. The NGO defines its own activity types and what data each one collects.

3. Resources — Money, Materials, Time

This covers grants and funds (with restricted vs. unrestricted tracking), inventory (medicines, textbooks, kits), and human effort (staff hours, volunteer hours). Resources flow *through* activities — “■50,000 from Grant X was used in Activity Y serving Entities Z.”

4. Indicators — Measurable Signals

The NGO's own metrics, attached at any level — a single entity, an activity type, a geography, or the whole program. The system does not prescribe *what* to measure; it gives scaffolding to define indicators, set targets, and auto-compute from activity data where possible.

5. Workflows — State Machines for Process

An application moves from submitted → reviewed → approved → disbursed. A grievance goes from received → assigned → resolved → closed. NGOs define their own workflows with custom states, transitions, permissions, and triggers.

6. Groups — Flexible Organizational Structure

The way NGOs organize their work is itself diverse, so it needs the same flexibility as every other primitive. A Group is a nestable, typeable organizational unit. The NGO defines the types (Programme, Center, Project, Initiative) and the hierarchy. Detailed in Section 4.

4. Groups: Flexible Organizational Structure

If we say “don’t force NGOs into a prescribed data model,” then hardcoding “Program” as *the* organizational unit contradicts the philosophy. The way NGOs organize their work is itself diverse — so we make it a flexible, typeable, nestable primitive called a **Group**.

How Groups Work

A Group is a named organizational unit. Every Group has a **type** that the NGO defines, and Groups nest inside each other according to rules the NGO sets. The word “Group” itself is mostly invisible to end users — they see their own vocabulary.

Example Hierarchies

NGO Type	Hierarchy	Example
Development NGO	Programme → Project → Activities	Maternal Health Programme → USAID FY26 Project → Trainings
Multi-center NGO	Center → Programme → Activities	Rajasthan Center → Livelihood Programme → SHG Meetings
Advocacy Org	Initiative → Campaign → Workstream → Activities	Climate Initiative → COP31 Campaign → Policy Track

The Three-Layer Naming Model

Config layer (admin sees): “Group Types” — the abstract concept. Define what kinds of organizational groupings exist and how they nest.

App layer (staff sees): The type name itself — Programme, Center, Project. The word “Group” is invisible.

Conversation layer (how people talk): “Which project is this under?” “Move it to the Rajasthan center.” Completely natural.

Structural Properties

Each Group type carries its own configurable metadata. A “Project” might have a start date, end date, donor, and budget ceiling. A “Center” might have a geography, head-of-center, and operating costs. Resources attach at whatever level makes sense — a grant might fund a whole Programme or just one Project within it. Indicators roll up through the hierarchy. Workflows can be scoped: “approval required at Centre level before disbursement.”

The data model is essentially a DAG — mostly a tree, but with flexibility for an Activity to belong to multiple Groups if needed (e.g., a joint training run by two Projects). This naturally solves **multi-donor reporting**: a single Activity sits inside multiple Groups, each tagged to a funding stream, giving clean fund attribution without duplicating data.

5. Tags: Cross-Cutting Descriptors

Tags and Groups solve different problems and should not be conflated.

Groups are Structural

They define *where work lives* in the organization. They have hierarchy, carry metadata, and govern behavior — permissions, reporting rollups, and workflows are scoped to Groups. Deleting a Group requires deciding what happens to everything inside it. They answer: **“How is our work organized?”**

Tags are Descriptive

They are labels you attach to anything to make it findable and filterable. They are flat, carry no metadata of their own, and do not affect behavior. Deleting a tag changes nothing structurally. They answer: **“What is this thing about?”**

***Example:** An NGO runs a joint training on nutrition for pregnant women in Rajasthan. The **Group path** is: Maternal Health Programme → USAID FY26 Project → Rajasthan Center. That determines budget, approvals, and reporting. The **Tags** are: nutrition, training, pregnant-women, field-activity. Those enable cross-cutting searches across the entire Group hierarchy.*

Tags are a first-class feature but not a “primitive” in the same structural sense — they are lightweight connective tissue that cuts across the organizational hierarchy.

6. The Composition Layer

This is where the primitives become powerful. The NGO wires them together into configured bundles within their Group hierarchy. Each Group contains a set of entity types, activity types, resources, indicators, and workflows that define how that part of the organization works.

Example: Livelihood Programme

Entities: Farmer, Self-Help Group | **Activities:** Training, Input Distribution, Market Linkage
Resources: NABARD Grant, Seed Kits | **Indicators:** Income Change, Adoption Rate
Workflows: Farmer Onboarding, Grant Disbursement

Example: Education Programme

Entities: Student, School, Teacher | **Activities:** Classroom Session, Assessment, Parent Meeting
Resources: USAID Grant, Textbooks | **Indicators:** Attendance, Learning Outcomes
Workflows: Student Enrollment, Teacher Training Certification

Same engine, completely different configurations. An NGO running both simply sets up two top-level Groups on the same platform.

7. The Form Builder

The form builder is the nervous system of NGO OS. Every activity, every entity update, every resource movement enters the system through a form. It is not a standalone feature — it is the primary interface between the field and the database.

Key Insight: Forms are not standalone. They are bound to primitives. A form submission does not just collect data — it creates Activities, updates Entities, deducts Resources, feeds Indicators, and triggers Workflows. One submission, many system actions.

Form Types

Form Type	What It Does	Example
Activity Form	Creates an Activity, links Entities, deducts Resources, logs location	"Training Session" form creates activity, marks attendees, increments indicator
Entity Form	Creates or updates an Entity with new data	"Farmer Registration" creates a Farmer; "Follow-Up Visit" updates crop yield
Resource Form	Records a resource movement and links to beneficiaries	"Kit Distribution" deducts inventory, links kits to receiving families

Layer 1: Field Design

The visual, drag-and-drop part — what the user filling the form sees. It should feel as easy as Google Forms but support the complexity field teams need.

- **Standard Fields** — Text, number, date, select, multi-select, photo, GPS, signature.
- **Entity Lookup** — Searches existing entities. “Which farmer are you visiting?” pulls up a searchable list filtered to the field worker’s assigned area. Works offline.
- **Repeat Groups** — “Add another attendee.” A single training form can register 30 participants without 30 separate submissions. Each repeat links to an entity.
- **Conditional Logic** — Branching based on earlier answers *or data already in the system*. If this farmer’s last visit was >90 days ago, show re-engagement questions.
- **Calculated Fields** — BMI from height and weight. Age from date of birth. Dosage from body weight. Computed in real-time on the device.
- **Validation Rules** — Date cannot be in the future. Phone must be 10 digits. GPS must be within 5km of assigned village (prevents fake submissions).

Layer 2: Data Binding

This is what most form tools completely lack, and where NGO OS earns its keep. The form builder has a binding interface where the admin maps form fields to system actions:

- **Field → Entity Property:** “Current Income” field updates the Farmer entity’s `income_current` property. Every follow-up visit automatically maintains an up-to-date profile.
- **Form → Activity Creation:** Submitting this form creates a “Home Visit” activity, linked to the selected Farmer, under the field worker’s assigned Group.
- **Field → Indicator Feed:** “Children Vaccinated” feeds into the “Vaccination Coverage” indicator for this geography.
- **Field → Resource Deduction:** “Kits Distributed” count deducts from the inventory of the linked warehouse entity.
- **Field → Workflow Trigger:** If “Referred to Hospital” is checked, trigger the “Referral Follow-Up” workflow, creating a task for the health coordinator.

The admin sees this as a simple mapping interface: form fields on the left, “what does this field do?” dropdowns on the right. Most forms need 2–3 bindings; complex ones might have 8–10.

Layer 3: Offline-First Runtime

Field workers open the app, see their assigned forms and a local entity database. Forms are filled and saved locally with timestamps and GPS. When connectivity returns, everything syncs. Conflicts use last-write-wins by default, with configurable merge rules for sensitive fields.

The local entity cache is smart — a field worker assigned to three villages gets those villages' entities synced to their device. They can look up and link to existing records even without internet. New entities created offline get temporary IDs that resolve to permanent ones on sync.

Form Versioning & Migration

Programs evolve. Six months in, the NGO wants to add a field to the training form. What happens to the 500 existing submissions? The form builder handles this gracefully: new fields are nullable for old records, removed fields are archived not deleted, and there is a clear version history so you can always see what the form looked like when a particular submission was made.

Templates & Sharing

An NGO designs a great antenatal care form. They publish it to a shared library. Another NGO doing maternal health imports it and tweaks it for their context. Over time, this builds a commons of field-tested form templates across the sector — a network effect most NGO tools cannot generate.

***In summary:** The form builder is three things in one — a **visual field designer** (drag-and-drop UX), a **data binding engine** (connecting fields to system primitives), and an **offline-first runtime** (the mobile app field teams use). All three must be excellent and work together seamlessly.*

8. Platform Services

Horizontal capabilities every NGO needs, regardless of domain.

Geo & Offline-First

Field teams work in low-connectivity areas. The system works offline on affordable Android devices and syncs when connectivity returns. Every entity and activity is geo-tagged.

Roles & Permissions

Field worker sees their village. Block coordinator sees their block. Program manager sees everything. Donor sees anonymized/aggregated data. Maps to the NGO's actual org structure.

Reporting & Dashboards

Auto-generated from indicators and activity data. Supports donor-specific reporting templates because every donor wants a different format — a massive sector pain point.

Compliance & Audit Trail

FCRA tracking for Indian NGOs, 501(c)(3) for US ones, GDPR for European operations. Every change is logged. Fund utilization certificates generated automatically.

Integrations & Open APIs

Connects to accounting software (Tally), payment gateways (for direct benefit transfers), government databases (Aadhaar, ration cards), and communication tools (WhatsApp, SMS).

9. Competitive Landscape

Existing Approach	Problem	NGO OS Approach
Salesforce NPSP	Too complex, expensive, Western-centric	Built for low-resource, offline-first contexts
Custom Spreadsheets	Doesn't scale, no relationships between data	Structured but flexible data model
Vertical SaaS	Locks you into one domain (health, education, etc.)	Domain-agnostic primitives
ERP Systems	Forces NGO into corporate workflows	NGO defines its own workflows and organizational structure

10. Business Model & What's Next

An **open-core** model aligns with the social sector's ethos and donor expectations around sustainability. The core primitives and self-hosted version are open source. Revenue comes from hosted/managed services, premium analytics, implementation support, and enterprise features like SSO, advanced permissions, and multi-org tenancy.

The Make-or-Break Challenge

The hardest part is not the technology — it is the **configuration UX**. Making it easy for a non-technical program manager to define entity types, set up Group hierarchies, design forms with data bindings, configure indicators, and wire workflows — all without needing a developer. The system must feel simple on Day 1 yet remain powerful enough for Year 5.

***The goal:** Every NGO, regardless of size or domain, should have access to the kind of operational infrastructure that only the largest organizations can afford to build custom today.*